

IncidentMind: Cross-Topology Root Cause Analysis for Microservices with GraphSAGE, Learned Dispatch and LLM Agents

Rakshitha Yathiraj

Dept. of Computer Science & Engg.
B.M.S. College of Engineering
Bengaluru, India
rakshitha.raj004@gmail.com

Author Name

Dept. of Computer Science & Engg.
B.M.S. College of Engineering
Bengaluru, India
email address

Author Name

Dept. of Computer Science & Engg.
B.M.S. College of Engineering
Bengaluru, India
email address

Author Name

Dept. of Computer Science & Engg.
B.M.S. College of Engineering
Bengaluru, India
email address

Author Name

Dept. of Computer Science & Engg.
B.M.S. College of Engineering
Bengaluru, India
email address

Abstract—When a microservice fails, the fault spreads along service-call edges and on-call engineers must search dozens of noisy signals to find its origin. Graph-based root cause analysis (RCA) models localise faults well on the topology they were trained on, but are rarely tested on a topology they have never seen, and they stop at a ranked list rather than an investigation. We present IncidentMind, an RCA pipeline that couples (i) an inductive GraphSAGE scorer that ranks services by root-cause probability, (ii) a Proximal Policy Optimisation (PPO) policy that decides which service to investigate next under a step budget, and (iii) locally hosted LLaMA 3.2 3B agents that inspect logs, metrics and code changes and write a structured incident report. The scorer and policy are trained only on RCAEval RE1 (Online Boutique, 11 services) and evaluated without retraining on ShopMind, a 12-service e-commerce testbed we built with fault injection. On held-out RE1 incidents the scorer reaches PR@1 of 0.92 (peer z-score: 0.12). Transferred zero-shot to ShopMind it reaches PR@1 of 0.51 (random: 0.08), with 0.91 on CPU stress and 0.80 on memory pressure but only 0.08 on pod crashes, a gap we trace to the absence of error-rate signal in the training data. The learned dispatcher solves 57% of ShopMind incidents within five steps versus 51% for greedy dispatch (exact McNemar $p = 0.031$). Given the correct target service, the agent layer names the root cause in 82% of 100 incidents and abstains in 14%. We also report four live end-to-end fault-injection runs, including a simultaneous dual fault, and release the testbed and pipeline.

Keywords—root cause analysis, microservices, graph neural networks, GraphSAGE, reinforcement learning, LLM agents, AIOps

I. INTRODUCTION

Cloud-native applications are built from many small services that call each other over the network. This design makes teams faster, but it makes failures harder to diagnose: a CPU-starved authentication service can show up as timeouts in a gateway, and a slow payment service can make its caller look like the problem [1], [2]. Engineers must correlate metrics, logs, traces and recent changes across services before they can act.

Automated RCA has moved from statistical correlation and causal inference [3], [4] towards models that use the

service dependency graph directly [5]–[7]. More recently, LLM-based agents have been applied to incident investigation [8]–[11]. Two practical gaps remain. First, most graph models are trained and tested on the same application, so it is unclear whether they transfer to a new topology, which is the normal situation when a team adopts a tool; recent work even questions whether graph structure helps at all [12]. Second, a ranked list of suspects is not an investigation: someone still has to decide which service to inspect, gather evidence, and write up the finding.

IncidentMind addresses both. A GraphSAGE model [13] learns an aggregation function over neighbours rather than a fixed adjacency transform, so it can score a graph it has never seen. A PPO policy [14] reads the ranked scores and the investigation history and picks the next target, so a wrong first guess is not the end of the search. Specialist LLM agents then examine the chosen service and produce a report. The contributions of this paper are:

- **A three-layer RCA pipeline** (graph scoring, learned dispatch, LLM investigation) with fixed-size state and action spaces, so the same trained models run unchanged on topologies of different size.
- **A strict cross-topology evaluation**: models and feature normalisation are fitted only on RCAEval RE1 [15]; ShopMind is used only for testing.
- **ShopMind**, a 12-service containerised testbed with four injectable fault types, a 100-incident evaluation set with ground-truth labels, and a live inject-and-capture path.
- **A per-fault failure analysis**, including a failure mode (pod crashes) that we diagnose rather than patch, and operational extensions for multi-fault detection, business-priority re-ranking and multi-instance telemetry.

II. RELATED WORK

Statistical and causal RCA. Log parsers and detectors such as Drain [16] and DeepLog [17] find anomalous events but do not model service topology. Causal-inference methods

such as CIRCA [4] rank root causes from metric dependencies; a large benchmark study at ASE 2024 found that no causal method performs consistently well across systems and fault types [3]. The resulting RCAEval benchmark [15] provides the RE1 data we train on.

Graph-based fault localisation. MicroRank [5] applies spectrum analysis over traces, Eadro [6] learns jointly from logs, metrics and traces, and CHASE [7] builds a causal heterogeneous graph. These models are typically trained and tested on one application. Gao et al. [12] show that a plain MLP can match GNNs on existing multimodal benchmarks, which motivates testing whether graph models add value when the topology changes. We test transfer to an unseen topology explicitly.

LLM agents for incidents. RCACopilot [9] and the study by Roy et al. [8] show that LLMs can collect diagnostic data and suggest root causes in production settings. Flow-of-Action [10] guides a multi-agent system with standard operating procedures, and OpenRCA [11] finds that current LLMs still struggle to locate root causes from raw telemetry. IncidentMind uses a learned graph model to narrow the search before any LLM is called, and uses a small local model so no telemetry leaves the organisation.

Reinforcement learning in AIOps. RL has mainly been applied to autoscaling and resource management; surveys of incident management [18] report little work on learning *investigation order*. Our dispatcher is a small step in that direction.

III. PROBLEM FORMULATION

A microservice system at time t is a directed graph $G = (V, E)$, where V is the set of services and $(u, v) \in E$ means u calls v . Each service v has a feature vector $\mathbf{x}_v \in \mathbb{R}^5$: CPU usage, memory usage, mean latency, error rate and 99th-percentile latency. An incident $I = (G, \{\mathbf{x}_v\}, r^*)$ has one ground-truth root-cause service $r^* \in V$.

Localisation. A scorer f returns $p_v = f(G, \mathbf{X})_v$, the probability that $v = r^*$, and a ranking π of V by p_v . We report $\text{PR}@k = \mathbf{1}[r^* \in \pi_{1:k}]$ averaged over incidents (the top- k hit rate) and MTTD, the mean rank of r^* in π (the number of services an engineer would inspect in order).

Dispatch. Investigation is an episodic decision process. At step t the state s_t holds the ranked scores and the set of visited services; the action $a_t = (\text{agent}, \text{slot})$ picks an agent type and a rank slot. The episode ends when the target equals r^* (solved) or after a budget of $B = 5$ steps. The policy $\pi_\theta(a_t | s_t)$ should maximise the solve rate and minimise steps.

IV. INCIDENTMIND ARCHITECTURE

Fig. 1 shows the pipeline. Telemetry is captured as a snapshot, scored by GraphSAGE, passed to the PPO dispatcher, investigated by agents, and summarised by a report agent.

A. GraphSAGE Root-Cause Scorer

The scorer is a three-layer GraphSAGE network [13] with mean aggregation (widths $5 \rightarrow 64 \rightarrow 128$, ReLU, dropout 0.2) followed by a linear two-class head. Edges are added in both directions so that anomaly evidence can flow from a cause to its callers and back, since the direction

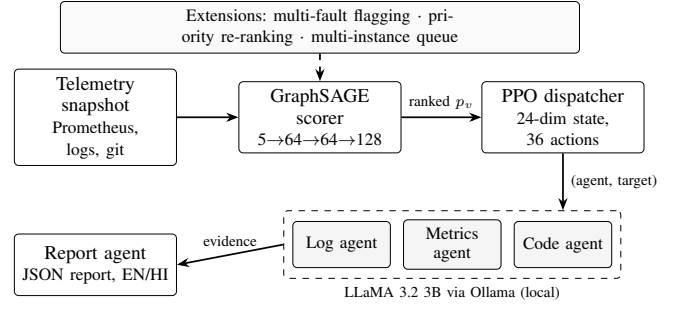


Fig. 1. IncidentMind pipeline. Only the scorer and dispatcher are learned; both are trained on RCAEval RE1 and applied unchanged to ShopMind.

of propagation is unknown at inference time. Features are z-normalised with per-feature means and standard deviations fitted once on the training split and stored with the checkpoint; they are never refitted on a test incident or on ShopMind. Without this step, raw feature scales (memory in bytes, error rate below one) saturated the logits and training stalled at a loss of $\ln 2$.

Training is node classification with class-weighted cross-entropy: the root-cause class is weighted by $|V| - 1$ to offset having one positive node per graph. We use Adam (learning rate 0.01) for 50 epochs. The score p_v is the softmax probability of the root-cause class. Because SAGE layers learn a function of a node's own features and its neighbours' mean, the same weights apply to any graph size or wiring.

B. Learned Dispatch with PPO

The observation is a fixed 24-dimensional vector: the scores of the top 12 rank slots (zero-padded) and a 12-bit visited mask in the same slot order. The action space has $3 \times 12 = 36$ discrete actions, one per (agent type, rank slot); slots beyond the graph size wrap modulo $|V|$. Defining actions over *rank slots* rather than service names is what lets one policy run on both the 11-node RE1 graph and the 12-node ShopMind graph.

The reward is $+1$ when the chosen service is r^* , -0.1 per step, and an extra -0.5 for revisiting a service; episodes end on success or after five steps. We train with Stable-Baselines3 [21] PPO (MLP policy and value networks [64, 64], rollout 2048, batch 64, 10 epochs, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2, entropy coefficient 0.01, learning rate 3×10^{-4} , 20,480 timesteps, seed 42) on the RE1 training split only, using the trained GraphSAGE scores as state.

C. Investigation Agents

Each agent is LLaMA 3.2 3B [19] served locally through Ollama at temperature 0, with a system prompt and a JSON output schema. The *log agent* reads recent container logs of the target; the *metrics agent* receives baseline-versus-failure-window averages and deltas for every service (the injected fault type is never included in any prompt); the *code agent* summarises recent commits touching the target. Each returns a finding, severity, confidence and evidence list. The agents read files from disk and are agnostic to whether the data came from a replay or a live capture.

D. Report Agent

The report agent merges the evidence bundle into a structured report: root-cause service, confidence, evidence summary, suggested fix, estimated blast radius, and report text in English and Hindi. It may output `unknown` when evidence is insufficient.

E. Operational Extensions

Three lightweight additions target deployment rather than accuracy. *Multi-fault flagging* lists every service whose score exceeds 0.5 instead of surfacing only the dispatch target. *Priority re-ranking* computes $\text{priority}_v = p_v \cdot w_v$ with static criticality tiers ($w = 1.0$ for auth, order, payment and primary database; 0.6 for medium services; 0.3 for search, notification and frontend); raw scores in the PPO state are left unchanged and only the order is adjusted. *Multi-instance telemetry* is a small HTTP receiver that stores snapshots tagged by instance, and a global queue keys nodes by (instance, service) so that identically named services on different deployments are not confused.

V. EXPERIMENTAL SETUP

A. Datasets

RCAEval RE1-OB (training domain). RE1 [15] contains 125 Online Boutique [20] incidents: five root-cause services (ad, cart, checkout, currency, product catalogue) $\times$ five faults (CPU, memory, disk, delay, packet loss) $\times$ five repetitions. Each incident becomes an 11-node, 15-edge graph using the published Online Boutique call graph, with features averaged over the post-injection window. We use a fixed 60/20/20 split (seed 42): 75 training, 25 validation, 25 test incidents. RE1 provides no error-rate metric, so that feature is constant zero in training.

ShopMind (unseen topology). ShopMind is a 12-service e-commerce application we built for this study: an Nginx gateway, a web frontend, seven Node.js services (auth, user, order, payment, inventory, notification, search), PostgreSQL primary and replica, and Redis, with 16 call edges. Services export Prometheus metrics and OpenTelemetry traces to Jaeger. Each application service exposes an `/inject-fault` endpoint for four faults: *CPU stress* (busy loop for 80ms of every 100ms), *memory pressure* (bounded heap growth), *network delay* (2s delay before handling), and *pod crash* (`process.exit(1)` re-triggered on restart until the fault window ends). After each fault, a settle check waits until $\text{CPU} < 0.15$, mean latency < 100 ms and error rate < 0.10 before the next incident, to avoid spill-over between incidents. The frozen evaluation set has 100 incidents over all 28 service-fault pairs (23 CPU, 25 memory, 27 delay, 25 crash). Each incident uses the peak-anomaly snapshot of its failure window; units are aligned to RE1 conventions at export time (CPU in percent, latency in seconds, memory in bytes).

B. Baselines

For localisation we compare against a *random* ranking and a training-free *peer z-score* scorer that ranks services by the root-mean-square z-score of their features relative to the other services in the same snapshot. For dispatch, all

TABLE I
ROOT-CAUSE LOCALISATION. RE1: 25 HELD-OUT TEST INCIDENTS (11 NODES). SHOPMIND: ALL 100 INCIDENTS (12 NODES), ZERO-SHOT WITH RE1-FITTED MODELS AND NORMALISATION.

Method	RE1 (in-domain)				ShopMind (unseen)			
	PR@1	PR@3	PR@5	MTTD	PR@1	PR@3	PR@5	MTTD
Random	0.09	0.27	0.45	6.00	0.08	0.25	0.42	6.50
Peer z-score	0.12	0.72	0.80	3.28	0.18	0.77	0.78	3.84
GraphSAGE	0.92	1.00	1.00	1.08	0.51	0.62	0.70	3.54

methods use identical GraphSAGE scores and the same five-step budget:

- **Baseline A (random):** uniform choice among unvisited services flagged anomalous ($p_v \geq 0.5$).
- **Baseline B (threshold):** first unvisited anomalous service in a per-incident shuffled arrival order, so neither score order nor node position leaks the answer.
- **Baseline C (greedy):** the top-ranked service only; it has no memory, so a wrong first pick is never corrected.

C. Metrics and Statistics

We report PR@1/3/5 and MTTD for localisation, and solve rate and mean steps (unsolved episodes counted at the budget) for dispatch. We give 95% bootstrap confidence intervals (10,000 resamples) for PR@ k , and use an exact McNemar test on paired per-incident outcomes to compare dispatchers. For the agent layer we report root-cause accuracy, abstention (`unknown`) rate and JSON validity.

D. Implementation

The pipeline uses Python 3.11, PyTorch, PyTorch Geometric [22] and Stable-Baselines3; ShopMind runs under Docker Compose on a single laptop. All learned numbers below come from one pair of released checkpoints; the unit-test suite checks the data contracts, dispatch baselines and cross-instance queue.

VI. RESULTS

A. In-Domain Localisation (RE1 Test Split)

Table I (left columns) shows that GraphSAGE ranks the true root cause first in 23 of 25 held-out RE1 incidents ($\text{PR}@1 = 0.92$, 95% CI [0.80, 1.00]) and within the top three in all 25 ($\text{MTTD} = 1.08$). The peer z-score scorer usually places the root cause in the top three (0.72) but rarely first (0.12): on 21 of the 25 test incidents it ranks the busy entry service *frontend* first, because its load deviates more from its peers than the faulty service does. Per fault, GraphSAGE reaches $\text{PR}@1 = 1.00$ on CPU, memory, disk and loss, and 0.33 on delay (3 incidents), where it still ranks the cause within the top three.

B. Cross-Topology Transfer (ShopMind)

Applied without retraining to ShopMind, which differs from RE1 in service count, wiring, implementation language and fault mechanisms, GraphSAGE achieves $\text{PR}@1 = 0.51$ ([0.41, 0.61]), six times the random rate and almost three times the z-score scorer (0.18). Fig. 2 breaks this down by fault. Transfer is strong for faults that raise a resource metric on the faulty service itself: CPU stress (0.91) and memory

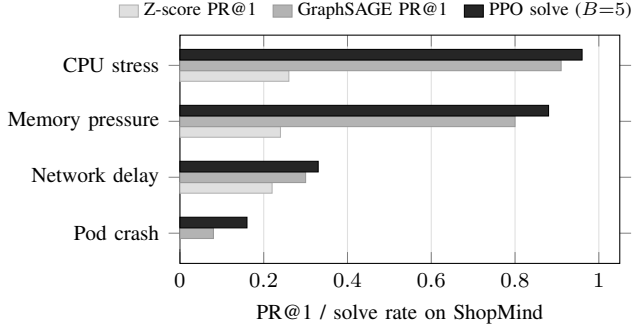


Fig. 2. Per-fault zero-shot results on ShopMind ($n = 23/25/27/25$ for CPU/memory/delay/crash).

pressure (0.80). It is weak for network delay (0.30) and pod crash (0.08).

The two failures have different causes. *Network delay* makes the *caller* look worst: in ShopMind, when *payment* is delayed, *order* accumulates pending requests and shows latency above 1.2s and timeout errors, while the delayed service can report near-zero span durations because clients disconnect before it responds. A scorer that follows symptoms is pulled one hop upstream. *Pod crash* leaves the crashed service with zeroed resource and latency metrics and an error rate of 1.0, while its caller (often *order*) shows raised latency and errors. Error rate is constant zero throughout RE1, so the model never learned that signal, and a zeroed node looks *less* anomalous than its busy neighbours; it is also indistinguishable in the other four features from uninstrumented nodes such as the gateway. Notably, the untrained z-score scorer ranks crashed services in the top three in 96% of crash incidents, because an error rate of 1.0 next to zeroed metrics stands out against its peers even without training. This suggests that the two signals are complementary. We report this as a limitation of the training data instead of adding a hand-written crash rule, which would weaken the transfer claim.

C. Learned Dispatch

Table II compares dispatchers given identical scores. On RE1 the scorer is already accurate, so all methods solve almost everything; PPO needs the fewest steps among those that solve all 25 (1.08), and greedy dispatch fails the two incidents whose root cause was ranked second.

On ShopMind the differences are larger. PPO solves 57% of incidents within five steps, compared with 51% for greedy and 45% for the random and threshold baselines. On paired outcomes, PPO solves six incidents that greedy misses and greedy solves none that PPO misses (exact McNemar $p = 0.031$). Mean steps per incident fall from 2.96 to 2.78. The gains come from the harder faults: +0.08 on pod crash, +0.04 on network delay and +0.08 on memory pressure over greedy. Tracing the episodes shows what the policy learned: it always inspects rank 1 first and, after a miss, inspects rank 2 (all 57 solves occur at step 1 or 2). It does not explore beyond the second slot; in the remaining 43 episodes it revisits ranks 1–2 until the budget runs out, despite the revisit penalty. So its gain comes from a learned second guess, and it cannot recover root causes ranked third or

TABLE II
DISPATCH UNDER A FIVE-STEP BUDGET WITH IDENTICAL GRAPHSAGE SCORES. STEPS: MEAN OVER ALL INCIDENTS, UNSOLVED COUNTED AS 5.

Dispatcher	RE1 test ($n = 25$)		ShopMind ($n = 100$)	
	Solve	Steps	Solve	Steps
A: random (anomalous)	1.00	1.12	0.45	3.27
B: threshold, arrival	1.00	1.12	0.45	3.24
C: greedy top-1	0.92	1.32	0.51	2.96
PPO (ours)	1.00	1.08	0.57	2.78

TABLE III
AGENT-LAYER ACCURACY UNDER ORACLE DISPATCH (100 SHOPMIND INCIDENTS).

Fault	n	Correct	Unknown	Wrong
CPU stress	32	25 (78%)	7	0
Memory pressure	28	28 (100%)	0	0
Network delay	17	13 (76%)	3	1
Pod crash	23	16 (70%)	4	3
All	100	82 (82%)	14	4

lower. A richer state (e.g., score gaps or per-feature evidence from agents) is needed for deeper search. Baselines A and B are limited because on ShopMind only 0.78 services per incident cross the 0.5 threshold on average, so they often run out of candidates.

D. Agent Layer

To isolate the LLM layer from localisation errors, we dispatched the metrics and report agents to the ground-truth service on a separate run of 100 ShopMind incidents with raw telemetry series (oracle dispatch; fault type withheld from prompts). Table III summarises the results. The report named the correct root cause in 82 incidents, output *unknown* in 14, and named a wrong service in 4; all 100 reports were valid JSON. Accuracy was highest on memory pressure (28/28) and lowest on pod crash (16/23). All four wrong answers involved a fault on *notification-service* being attributed to *auth-service*. Stated confidence was informative: it averaged 0.69 on correct reports and 0.11 on incorrect or abstained ones. This is an upper bound for the agent layer, since the end-to-end pipeline passes on scorer and dispatcher errors.

E. Live End-to-End Runs

We ran the full pipeline on live ShopMind faults: inject, capture five baseline and 15 failure snapshots at 2s intervals, select the peak snapshot, score, dispatch, investigate and report. Table IV lists the four runs from the final integrated build. In each, GraphSAGE ranked the injected service first, PPO dispatched to it on the first step, and the report named it with confidence 0.8. In the dual-fault run (CPU stress on *order* and *inventory* at the same time), both services scored 1.00 and multi-fault flagging reported both, whereas the single dispatch target alone would have surfaced only one.

Two earlier live runs illustrate the extensions and the main weakness. In a CPU-stress run on *auth*, GraphSAGE gave *auth* and *notification* equal scores of 1.00; priority re-ranking

TABLE IV
LIVE END-TO-END RUNS ON SHOPMIND (FINAL BUILD).

Fault	Injected service(s)	Rank-1 score	PPO step	Report
CPU stress	auth	1.00	1	correct
CPU stress	order	1.00	1	correct
Network delay	payment	1.00	1	correct
Dual CPU	order + inventory	1.00 / 1.00	1	correct [†]

[†]Report names *order*; multi-fault flag lists both services.

kept *auth* (high tier) first and moved *notification* (low tier) down. In a live pod-crash run on *auth*, the scorer instead ranked *payment* first on a small CPU fluctuation (score 0.45), and the dispatcher, agents and report followed it with confidence 0.5. This matches the offline pod-crash result. On the 100-incident benchmark, priority re-ranking changed PR@1 from 0.51 to 0.48 and left PR@5 unchanged at 0.70. As intended, it reorders by business impact and does not improve accuracy.

VII. DISCUSSION AND LIMITATIONS

What transfers. The in-domain versus cross-domain gap (0.92 vs. 0.51 PR@1) is large, and part of the in-domain score probably comes from RE1’s fixed topology and only five root-cause services, which a model can partly memorise. The per-fault picture is more informative than the average: faults whose signature sits on the faulty node transfer almost fully, while faults whose signature appears on neighbours or in features absent from training do not. Graph structure alone does not solve cross-topology RCA. Training on sources with richer features, for example RCAEval RE2/RE3 with logs and traces, is the most direct fix.

Report quality. Root-cause attribution was reliable when evidence was clear, but free-text parts were weaker. Some suggested fixes repeated harmless log lines (e.g., a service “listening on a port”) as causes, one CPU-stress report mentioned pod crashes that did not occur, and the Hindi text was sometimes repetitive. These are typical of a 3B model; structured fields should be trusted over narrative text, and translation quality was not formally evaluated.

Threats to validity. The RE1 test split has only 25 incidents, so its confidence intervals are wide. ShopMind is a controlled testbed with four fault types and one instance per service. The agent evaluation used oracle dispatch and a separately generated incident set, so it is not a direct end-to-end accuracy figure. Live runs are case studies, not a benchmark. Multi-instance support was verified over loopback; a true cross-machine run and notification hooks are future work. RE1 features are window averages while ShopMind uses peak snapshots, which is an additional distribution shift.

VIII. CONCLUSION

IncidentMind combines an inductive GraphSAGE scorer, a PPO dispatcher over rank slots, and local LLM agents into a single RCA pipeline that runs unchanged on a topology it was not trained on. Trained only on RCAEval RE1, it localises root causes on the unseen ShopMind testbed at six times the random rate, with strong transfer on resource faults and a clearly diagnosed failure on crashes caused by missing training signal. Learned dispatch recovers significantly more

incidents than greedy dispatch by learning a second guess, and the agent layer turns a correct target into a correct, machine-readable report 82% of the time. Future work includes training on multimodal sources to close the crash and delay gaps, combining learned and statistical scores, and multi-machine deployment. The code and testbed are available at <https://github.com/rakshithay3/incident-mind>.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, for the facilities and support provided for this work. We also thank the authors of RCAEval for releasing their benchmark data publicly, and the maintainers of PyTorch Geometric, Stable-Baselines3 and Ollama, whose open-source tools made this study possible.

The authors used Claude (Anthropic), an AI assistant, to write the analysis scripts that computed the metrics in Tables I–II and Fig. 2 from the authors’ trained models and datasets. The IncidentMind system, the ShopMind testbed, model training, the agent evaluation and the live fault-injection runs are the authors’ own work. The authors reviewed and checked all AI-generated content and take full responsibility for the paper.

REFERENCES

- [1] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, “Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study,” *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, 2021.
- [2] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey,” *ACM Comput. Surv.*, vol. 55, no. 3, 2022.
- [3] L. Pham, H. Ha, and H. Zhang, “Root cause analysis for microservices based on causal inference: How far are we?” in *Proc. 39th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2024.
- [4] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, “Causal inference-based root cause analysis for online service systems with intervention recognition,” in *Proc. 28th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2022.
- [5] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, “MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments,” in *Proc. Web Conf. (WWW)*, 2021.
- [6] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, “Eadro: An end-to-end troubleshooting framework for microservices on multi-source data,” in *Proc. 45th Int. Conf. Software Engineering (ICSE)*, 2023.
- [7] Z. Zhao, T. Zhang, Z. Shen, H. Dong, X. Ma, X. Liu, and Y. Yang, “CHASE: A causal heterogeneous graph based framework for root cause analysis in multimodal microservice systems,” arXiv:2406.19711, 2024.
- [8] D. Roy, X. Zhang, R. Bhavé, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, “Exploring LLM-based agents for root cause analysis,” in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.
- [9] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, “Automatic root cause analysis via large language models for cloud incidents,” in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.
- [10] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li, G. Xie, and D. Pei, “Flow-of-Action: SOP enhanced LLM-based multi-agent system for root cause analysis,” in *Companion Proc. ACM Web Conf. (WWW)*, 2025.
- [11] J. Xu *et al.*, “OpenRCA: Can large language models locate the root cause of software failures?” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.
- [12] F. Gao, R. Xin, X. Li, and Y. Zhang, “Are GNNs actually effective for multimodal fault diagnosis in microservice systems?” arXiv:2501.02766, 2025.

- [13] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Kavukcuoglu, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- [15] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proc. ACM Web Conf. (WWW)*, 2025.
- [16] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017.
- [17] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017.
- [18] Y. Remil, A. Bendimerad, R. Mathonat, and M. Kaytoue, "AIOps solutions for incident management: Technical guidelines and a comprehensive literature review," arXiv:2404.01363, 2024.
- [19] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.
- [20] Google Cloud Platform, "Online Boutique: Cloud-native microservices demo application," GitHub repository. [Online]. Available: <https://github.com/GoogleCloudPlatform/microservices-demo>
- [21] A. Raffin, A. Hill, A. Gernhardt, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
- [22] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.